Community Guardian

Finalised Implementation Plan — v2

Palo Alto Networks — New Grad SWE Take-Home

Scenario	Option 3 — Community Safety & Digital Wellness
Timebox	~13 hours (deliberate — full production-grade stack)
Frontend	Next.js 14 App Router + Tailwind CSS + Zustand
Database	PostgreSQL via Neon (serverless) + Prisma ORM
AI	Anthropic Claude API — 2 calls (Categorize + NL Query)
Real Data	CISA KEV + PhishTank + NVD (official public APIs)
Deploy	Vercel + Neon free tier — zero config
Unique Features	Safety Pulse + Similar Alerts + Threat DNA Fingerprint + "This Affects Me"

Single source of truth. Follow execution order. □ phases are non-negotiable.

§1 What's New in This Version

Three major additions over the previous plan:

Addition	Impact	Time Added
Full DB Integration (Neon + Prisma)	Production-grade persistence. Alerts, preferences, guardians all stored in PostgreSQL. Survives refresh, device change, deploy.	**+~3.25 hours**
Threat DNA Fingerprint	Deterministic SVG generated per alert from category + severity + source + title hash. Every alert gets a unique visual identity.	**+~30 min**
"This Affects Me" One-Tap	On CISA/NVD alerts: one toggle marks personal relevance. Digest prioritises these. Safety Pulse counts them separately. Hits rubric contextual relevance metric directly.	**+~25 min**

Total revised timebox: ~13 hours. Noted in README as deliberate — full production architecture chosen over minimum viable submission.

§2 Requirements Audit — Both Documents

Case Study Brief (Doc 1) — Every Requirement

✓	Requirement	Status	How
✓	Core flow: Create + View + Update	**Met**	Paste + manual create, list/digest/detail views, edit, mark resolved
✓	Search + Filter	**Met**	Text search + category/severity/status/date/source/location/affects-me filters
✓	AI capability (Categorize)	**Met**	Call 1 → 5 fields in one call: category, severity, summary, action, reason
✓	Fallback when AI unavailable	**Met**	Keyword rules engine + urgency regex + confidence scoring + UI warning
✓	Input validation + clear errors	**Met**	Required fields, date validity, max lengths, inline + toast errors
✓	Minimum 2 tests	**Met**	4 tests: happy path, edge case, fallback, empty state
✓	Synthetic data — no live scraping	**Met**	data/alerts.json (40-50 rows) + official public APIs (not scraping)
✓	Sample CSV/JSON in repo	**Met**	data/alerts.json + data/category_checklists.json
✓	No API keys committed	**Met**	.env (gitignored) + .env.example (committed, placeholders only)
✓	Working prototype	**Met**	Deployed to Vercel with live demo link
✓	Design documentation	**Met**	DESIGN.md: problem, solution, architecture, AI design, future work
✓	Completed README	**Met**	Full template: AI disclosure, tradeoffs, limitations, live link
✓	5-7 min video	**Met**	Screen recording covering all rubric scoring points

Community Guardian Plan (Doc 2) — All Features

	Feature	Status
✓	Create alert — paste path (AI prefill)	Implemented — Phase 4
✓	Create alert — manual form with validation	Implemented — Phase 4
✓	View alerts list + skeleton loading	Implemented — Phase 3
✓	View alerts digest ("For you" / "All active")	Implemented — Phase 7

	Feature	Status
✓✓	Update / mark resolved / reopen / edit	Implemented — Phase 6
✓✓	Search & filter + clear filters	Implemented — Phase 3
✓✓	AI Call 1: structured JSON (5 fields)	Implemented — Phase 5
✓✓	Fallback: keywords + urgency + confidence + reason	Implemented — Phase 5
✓✓	"Verify this classification" UI warning	Implemented — Phase 5
✓✓	"Explain why" reason display (collapsible)	Implemented — Phase 6
✓✓	Static 1-2-3 category checklists	Implemented — Phase 6
✓✓	Safety Pulse ⋆ (unique)	Implemented — Phase 3
✓✓	Similar Alerts ⋆ (unique)	Implemented — Phase 4
✓✓	My Concerns onboarding	Implemented — Phase 7
✓✓	Digest view + Focus Mode	Implemented — Phase 7
✓✓	Calm UI + dark mode + reduced motion	Implemented — Phase 2
✓✓	Read aloud (speechSynthesis)	Implemented — Phase 8
✓✓	Guardian Circle + trusted contacts	Implemented — Phase 9
✓✓	Share digest / share alert (read-only links)	Implemented — Phase 9
✓✓	"Copy status message" for WhatsApp/SMS	Implemented — Phase 9
✓✓	Keyboard nav + ARIA + skip link	Implemented — Phase 8
✓✓	4 tests (happy, edge, fallback, empty state)	Implemented — Phase 10
✓✓	Deploy + live demo link	Implemented — Phase 11
✓✓	DESIGN.md + README	Implemented — Phase 11
+	Full DB — Neon + Prisma (alerts, prefs, guardians)	ADDED — Phase 1 + all CRUD phases
+	Real data: CISA + PhishTank + NVD (persisted to DB)	ADDED — Phase 1
+	Alert Source Badge (CISA/PhishTank/NVD/User)	ADDED — Phase 3
+	AI Call 2 — Natural Language Query	ADDED — Phase 8
+	Severity Trend Chart (Recharts)	ADDED — Phase 7
+	Location-based filtering	ADDED — Phase 8
+	"Last Checked" timestamp	ADDED — Phase 3

	Feature	Status
+	Threat DNA Fingerprint ⭑ (unique)	ADDED — Phase 3 + Phase 6
+	"This Affects Me" one-tap ⭑ (unique)	ADDED — Phase 6

§3 Locked Final Feature List

□ Must Build — Core (Phases 0–6, 10–11)

#	Feature	Details
1	**Create Alert — Paste Path**	Paste text → AI Call 1 → prefill form → confidence badge → Similar Alerts → user edits → save to DB
2	**Create Alert — Manual Form**	title*, description*, category*, severity*, date*, location (optional) — all validated, saved to DB
3	**View Alerts List + Skeleton**	Sorted newest first, AlertCard per item (with Threat DNA badge), skeleton while loading
4	**Search & Filter**	Text search + category / severity / status / date / source / location / affects-me filters + clear all
5	**Update / Mark Resolved / Reopen**	Edit any field → PATCH /api/alerts/[id] → DB update → Zustand cache update
6	**Full DB — Neon + Prisma**	PostgreSQL via Neon serverless. Prisma ORM. Alerts + UserPreference + Guardian models.
7	**Real Data Layer**	CISA KEV + PhishTank + NVD via Next.js API routes. Normalised + persisted to DB (deduped by id).
8	**Alert Source Badge**	CISA=blue, PhishTank=orange, NVD=purple, User=slate — on every AlertCard and Detail page
9	**Threat DNA Fingerprint ★**	Deterministic SVG per alert: shape=category, colour=severity, pattern=source, variation=title hash
10	**Safety Pulse ★**	"You're caught up" (0 active) or "N things need attention" + "X directly affect you" if affects_me flagged
11	**AI Call 1 — Structured Categorize**	One call → JSON: category + severity + summary + suggested_action + reason
12	**Fallback Engine**	Keyword rules per category + urgency regex + confidence low + reason string
13	**"Verify this classification" warning**	Shown when confidence=low, prompts manual review before saving
14	**"Explain Why" reason display**	Free from AI Call 1. Shown on card/detail, collapsible.
15	**Category Checklists (1-2-3)**	Static steps per category from category_checklists.json — shown on Detail page
16	**Input validation + error messages**	Required fields, date validity, max lengths, inline + toast, "Using fallback" banner
17	**4 Tests**	Happy path, edge case (empty/invalid input), fallback (mock API fail), empty state

#	Feature	Details
18	**.env + .env.example**	ANTHROPIC_API_KEY + DATABASE_URL — never committed, placeholders in example
19	**data/alerts.json + category_checklists.json**	40-50 synthetic alerts + 1-2-3 steps per category — in repo for rubric + seeding

□ Strong — Goated Experience (Phases 7–9)

#	Feature	Details
20	**My Concerns Onboarding**	"What matters to you?" multi-select pills → saved to UserPreference table via /api/preferences
21	**Digest View**	"For you" (filtered by concerns from DB) + "All active" tabs
22	**Focus Mode**	One alert at a time — Next / Mark Done / Back — progress "2 of 7"
23	**Similar Alerts ★**	After paste+categorize: 1-2 "Similar to: [title]" via findSimilarAlerts(category, keywords, dbAlerts)
24	**"This Affects Me" One-Tap ★**	CISA/NVD alerts only: toggle → PATCH affects_me=true → DB → Digest prioritises → Safety Pulse counts separately
25	**AI Call 2 — Natural Language Query**	"What phishing scams are active?" → Claude reads real alert context from DB → calm plain-English answer
26	**Severity Trend Chart**	Recharts BarChart from DB query — last 14 days, stacked by severity, toggle by category/source
27	**Location-based Filtering**	Dropdown of unique location labels from DB, filters alert list, hits contextual relevance rubric metric
28	**"Last Checked" Timestamp**	"Data last updated: 3 mins ago" + refresh button — real data freshness signal
29	**Guardian Circle**	Add/edit/remove 2-5 trusted contacts (name + label) — stored in Guardian table via /api/guardians
30	**Share Digest / Share Alert**	Read-only links: /shared/digest/[token] + /shared/alert/[id] — copy to clipboard + "Copied!" toast
31	**"Copy Status Message"**	One-tap: "I have N active alerts (X directly affect me). View my digest: [link]"
32	**Calm UI + Dark Mode + Reduced Motion**	Soft palette, prefers-reduced-motion, system theme + manual toggle, persisted in UserPreference
33	**Read Aloud (TTS)**	speechSynthesis on detail/digest — title + summary + suggested action
34	**Keyboard Nav + ARIA + Skip Link**	Full a11y: focus management, aria-labels, semantic HTML, WCAG AA contrast

□ Bonus — If Ahead of Schedule

#	Feature	Est. Time	Details
35	**"I'm Good" One-Tap**	**~20 min**	After checklist complete: copies "I saw the [category] alert and followed the steps. I'm good."
36	**Calm Score**	**~20 min**	1-10 resolution-progress score — "how in control am I" not "how dangerous is it"
37	**Quiet Hours**	**~30 min**	Suppress non-critical alerts during user-set time window
38	**Keyboard Shortcut Sheet**	**~15 min**	"?" key opens shortcuts modal — small touch, big a11y signal

✕ Explicitly Out of Scope

Item	Reason
User authentication / accounts	No scoring benefit, adds significant auth complexity
Live site scraping	Rubric violation — we use official public APIs instead
Real encryption for Guardian Circle	localStorage share links sufficient per plan §15
Real GPS / location data	Synthetic location labels only per plan §15
Push notifications / Service Worker	Too complex, no scoring benefit in timebox
Multi-language / i18n	Irrelevant to rubric scoring criteria

§4 Tech Stack & Database

Full Stack

Layer	Choice	Why
Frontend	**Next.js 14 App Router**	Built-in API routes — proxy real data + AI calls without exposing keys
Styling	**Tailwind CSS**	Fastest path to calm, consistent design system
State / Cache	**Zustand**	Optimistic UI layer on top of DB — fast reads, consistent writes
Database	**PostgreSQL via Neon**	Serverless Postgres, free tier, works perfectly with Vercel, zero config
ORM	**Prisma**	Type-safe, auto-generates TypeScript types from schema, Next.js native
AI	**Anthropic Claude API**	Superior structured JSON output + calm tone — fits product philosophy
Real Data	**CISA KEV + PhishTank + NVD**	Official government/community APIs — real threat intelligence, no scraping
Charts	**Recharts**	React-native, lightweight, works with Tailwind dark mode
Tests	**Vitest + React Testing Library**	Fast, ESM-native, seamless with Next.js
Deploy	**Vercel + Neon free tier**	Zero config for Next.js + Postgres — live URL in minutes

Prisma Schema (prisma/schema.prisma)

```
generator client {   provider = "prisma-client-js" }  datasource db {   provider = "postgresql"   url
= env("DATABASE_URL") }  model Alert {   id               String   @id @default(uuid())   title
String   description      String   category         String   severity         String   summary
String?   suggested_action String?   reason           String?   confidence       String
@default("high")   source           String   @default("User")   location         String?   date
String   resolved         Boolean  @default(false)   affects_me       Boolean  @default(false)
created_at       DateTime @default(now())    @@index([category])   @@index([severity])
@@index([resolved])   @@index([source])   @@index([affects_me])   @@index([created_at]) }  model
UserPreference {   id            String   @id @default("default")   concerns      String[]
@default([])   theme         String   @default("system")   quiet_start   String?   quiet_end
String?   updated_at    DateTime @updatedAt }  model Guardian {   id         String   @id
@default(uuid())   name       String   label      String?   created_at DateTime @default(now()) }
```

Environment Variables

```
# .env (gitignored — never commit) ANTHROPIC_API_KEY=sk-ant-...
DATABASE_URL=postgresql://user:pass@host/db?sslmode=require  # .env.example (commit this)
ANTHROPIC_API_KEY=your_anthropic_api_key_here
DATABASE_URL=your_neon_postgresql_connection_string_here
```

Neon DB Setup (5 min)

```
1. Go to neon.tech → New Project → copy connection string 2. Paste into .env as DATABASE_URL 3. npx
prisma db push          # creates tables 4. npx prisma db seed          # inserts 40-50 synthetic
alerts 5. Done — DB is live
```

Alert Type (src/types/alert.ts)

```
export type AlertSource   = 'CISA' | 'PhishTank' | 'NVD' | 'User' export type AlertCategory = 'Scam'
| 'Phishing' | 'Imposter' | 'Data breach' | 'Local safety' | 'CVE' | 'Other' export type Severity
= 'low' | 'medium' | 'high' | 'critical' export type Confidence    = 'high' | 'low'  export interface
Alert {   id:               string   title:            string   description:      string   category:
AlertCategory   severity:         Severity   summary:          string   suggested action: string
reason:           string   confidence:       Confidence   source:           AlertSource   location?:
string   date:             string   resolved:         boolean   affects me:       boolean
created_at:       string }
```

§5 Database API Routes

All CRUD goes through Next.js API routes → Prisma → Neon. Components never touch the DB directly — they talk to Zustand (cache) which syncs with these routes.

Route	Method(s)	Purpose
/api/alerts	**GET + POST**	List all alerts (with filters via query params) + create new alert
/api/alerts/[id]	**GET + PATCH + DELETE**	Get one alert + update fields + delete
/api/preferences	**GET + PATCH**	Get/update UserPreference (concerns, theme, quiet hours)
/api/guardians	**GET + POST**	List all guardians + add new guardian
/api/guardians/[id]	**DELETE**	Remove a guardian
/api/feeds/cisa	**GET**	Fetch CISA KEV → normalise → upsert to DB → return
/api/feeds/phishtank	**GET**	Fetch PhishTank → normalise → upsert to DB → return
/api/feeds/nvd	**GET**	Fetch NVD CVEs → normalise → upsert to DB → return
/api/categorize	**POST**	AI Call 1 proxy — Anthropic Claude structured categorize
/api/query	**POST**	AI Call 2 proxy — NL query with DB alert context

GET /api/alerts — Query Parameters

Param	Behaviour
category=Phishing	Filter by exact category
severity=high	Filter by severity
status=active|resolved	Filter resolved boolean
source=CISA	Filter by source
affects_me=true	Filter affects_me=true only
location=Downtown	Filter by location label
search=gmail	Full-text search on title + description (case insensitive)
limit=50&offset=0	Pagination — default limit 50

Zustand Store — Cache Strategy

```
// Zustand is a CLIENT-SIDE CACHE on top of DB // Source of truth = Neon DB // Pattern: optimistic
update → API call → confirm or rollback  addAlert: async (data) => {   const optimistic = { ...data,
id: uuid(), created_at: now() }   set(state => ({ alerts: [optimistic, ...state.alerts] }))   //
optimistic   const saved = await POST("/api/alerts", data)   set(state => ({ alerts:
state.alerts.map(a =>     a.id === optimistic.id ? saved : a                        // confirm   }))
},  updateAlert: async (id, updates) => {   set(state => ({ alerts: state.alerts.map(a =>     a.id
=== id ? { ...a, ...updates } : a                     // optimistic   )))   await
PATCH(`/api/alerts/${id}`, updates)                    // confirm },
```

§6 AI Design

Call	Trigger	Output	Fallback
Call 1 Structured Categorize	User pastes text on Create page → clicks "Categorize"	JSON: category, severity, summary, suggested_action, reason	Keyword rules engine + urgency regex + confidence="low" + "Verify" warning
Call 2 NL Query	User types natural language question in ThreatQuery box	Calm plain-English answer drawn from last 20 DB alerts as context	"Search unavailable. Browse alerts below." + show FilterBar

AI Call 1 — System Prompt

```
You are a calm, factual safety assistant. Analyse the text and return ONLY valid JSON — no preamble, no text outside the JSON.  Categories: Scam, Phishing, Imposter, Data breach, Local safety, CVE, Other Severities: low, medium, high, critical  Return this exact shape: {   "category": "...", "severity": "...",   "summary": "One calm sentence under 120 chars.",   "suggested_action": "1-2 concrete steps.",   "reason": "Brief explanation why this category (under 80 chars)." }
```

One API call → five fields. We get Categorize + Summarize + Extract + Explain in a single call. Maximum output, minimum cost, minimum latency.

Fallback Keyword Map

Category	Trigger Keywords
Phishing	phishing, click here, verify account, login link, credentials, confirm your, update your password
Scam	prize, winner, send money, wire transfer, lottery, gift card, congratulations, claim your
Imposter	irs, social security, microsoft support, police, government, impersonat, official notice
Data breach	breach, leaked, compromised, stolen data, exposed, database, personal information, unauthorised access
Local safety	robbery, fire, accident, missing person, shooting, flood, evacuation, incident report
CVE	cve-, vulnerability, exploit, patch, zero-day, rce, remote code execution, security advisory

Urgency regex: /urgent|asap|immediately|right now|act now|time sensitive/i → severity bumped to "high"
Confidence always "low" for fallback → "Verify this classification" warning always shown

§7 Unique Features — Deep Dive

These four features do not appear anywhere in the case study brief. They are original product ideas.

1. Safety Pulse ★

A single at-a-glance status line at the top of every page. Answers "how am I doing?" not "what are the alerts?"

State	Display
0 active alerts	"✓ You're caught up" — soft green, calm language
N active, none affect me	"N things need your attention" — soft amber
N active, X affect me	"N things need your attention — X directly affect you" — amber with sub-line

2. Similar Alerts ★

After paste + AI categorize, shows 1-2 alerts from the DB corpus that match by category + keyword overlap. Reduces "is this real?" anxiety by linking new reports to known historical patterns.

```
// src/lib/similarity.ts export function findSimilarAlerts(   category: AlertCategory,   pastedText:
string,   alerts: Alert[],   limit = 2 ): Alert[] {   const keywords = pastedText.toLowerCase()
.split(/\W+/).filter(w => w.length > 4)     .filter(w => !STOPWORDS.includes(w))    return alerts
.filter(a => a.category === category && a.id !== "new")     .map(a => ({       alert: a,       score:
keywords.filter(k =>         (a.title + " " + a.description).toLowerCase().includes(k)       ).length
}))     .filter(x => x.score > 0)     .sort((a, b) => b.score - a.score)     .slice(0, limit)
.map(x => x.alert) }
```

3. Threat DNA Fingerprint ★

A deterministic SVG generated from alert attributes. Same alert always produces the same fingerprint. Makes the list visually scannable — threats are recognisable at a glance.

Attribute	Visual Mapping
category	Base shape: Phishing=circle, Scam=diamond, Imposter=hexagon, Data breach=square, CVE=triangle, Local safety=shield, Other=dot
severity	Colour intensity: low=light, medium=mid, high=saturated, critical=dark+bold border
source	Pattern overlay: CISA=horizontal lines, PhishTank=diagonal lines, NVD=grid, User=dots
title hash	Slight rotation (0-15°) + size variation (±2px) — makes each alert unique

```
// src/lib/fingerprint.ts export function generateFingerprint(alert: Alert): string {   const hash =
simpleHash(alert.title)    // deterministic 0-100   const shape = SHAPES[alert.category]   const
color = COLORS[alert.severity]   const pattern = PATTERNS[alert.source]   const rotation = hash % 15
const size = 20 + (hash % 4)    return `<svg viewBox="0 0 ${size} ${size}" ...>
<defs>${pattern}</defs>     <${shape} fill="${color}" transform="rotate(${rotation})" .../>
<${shape} fill="url(#pattern)" opacity="0.4" .../>   </svg>` }  function simpleHash(str: string):
number {   return str.split("").reduce((h, c) =>     ((h << 5) - h + c.charCodeAt(0)) & 0xff, 0   ) }
```

Where it appears: 24x24 thumbnail on every AlertCard. 64x64 hero on Detail page. No API call — pure deterministic math.

4. "This Affects Me" One-Tap ⭑

On CISA and NVD alert detail pages — a single toggle. When tapped, the alert is personally flagged in the DB. This directly hits the rubric's "Contextual Relevance" success metric.

Where it shows	Behaviour
Detail page (CISA/NVD only)	Toggle button: "This affects me" / "Doesn't affect me" → PATCH affects_me to DB
Safety Pulse	Counts "affects me" alerts separately: "2 directly affect you"
Digest view	"Directly affects you" section pinned to top of digest above general alerts
FilterBar	New filter: "Affects me" — shows only personally flagged alerts
"Copy status message"	Includes count: "Including 2 that directly affect my devices"

§8 Screens & Flows

Screen / Route	Contents
/alerts (Home / List)	Safety Pulse. "Last checked" + refresh. FilterBar (all filters incl. affects-me). Text search. Severity Trend Chart (collapsible, from DB). AlertCard list — each with Threat DNA fingerprint thumbnail + source badge. Skeleton while DB fetches. Empty state.
/digest	Safety Pulse. "For you" tab (My Concerns from DB) + "All active" tab. "Directly affects you" section pinned top (affects_me=true). Focus Mode toggle → FocusMode component. "Share digest" → read-only link. "Read digest aloud" → speechSynthesis queue.
/create?tab=paste	Two tabs. Paste textarea → "Categorize" → loading → AI Call 1 → prefill → confidence badge → "Verify" warning (if low) → SimilarAlerts component (1-2 from DB) → edit fields → Save → POST /api/alerts → DB → navigate to detail.
/create?tab=manual	Manual form. All fields with inline validation on blur + submit validation. Save → POST /api/alerts → DB → navigate to detail.
/alerts/[id] (Detail)	Large Threat DNA fingerprint. Title + all badges. AI summary. "Why this category?" (collapsible). Full description. CategoryChecklist (1-2-3, local checkboxes). "This Affects Me" toggle (CISA/NVD only) → PATCH DB. Suggested action. Read Aloud button. "Mark resolved" / "Reopen" → PATCH DB. "Edit". "Share this alert". External link for CISA/NVD.
/alerts/[id]/edit	Same AlertForm reused, pre-populated from DB. Save → PATCH /api/alerts/[id]. Cancel → back to detail.
/settings	My Concerns multi-select → PATCH /api/preferences. Guardian Circle (CRUD → /api/guardians). Theme toggle → PATCH /api/preferences. Reduced motion toggle.
/shared/digest/[token]	Read-only. No nav, no edit. Active alerts from DB. "Shared safety digest — view only". CategoryChecklists visible.
/shared/alert/[id]	Read-only. Single alert from DB. Title, summary, checklist, suggested action. "This alert was shared with you via Community Guardian."

§9 File Structure

```
/ ├── .env                              # gitignored ├── .env.example                      #
committed — placeholders ├── README.md ├── DESIGN.md ├── next.config.js ├── tailwind.config.js ├──
vitest.config.ts ├── prisma/ │   ├── schema.prisma                # Alert + UserPreference +
Guardian │   └── seed.ts                        # seeds data/alerts.json to DB ├── data/ │   ├──
alerts.json                    # 40-50 synthetic alerts (rubric req + seed) │   └──
category_checklists.json      # 1-2-3 steps per category ├── public/ │   └── favicon.ico └── src/
├── types/      │   └── alert.ts                  # Alert, AlertSource, AlertCategory, Severity,
Confidence      ├── lib/     │   ├── db.ts                      # PrismaClient singleton     │   ├──
ai.ts                      # getStructuredCategory() — AI Call 1     │   ├── fallback.ts
# fallbackCategorize(), keyword map, urgency regex     │   ├── similarity.ts             #
findSimilarAlerts(category, text, alerts, limit)     │   ├── fingerprint.ts            #
generateFingerprint(alert) → SVG string     │   └── constants.ts              # categories, severity
colours, source colours     ├── store/     │   └── useStore.ts               # Zustand — optimistic
cache over DB     ├── hooks/     │   ├── useAlerts.ts              # CRUD helpers: fetch, create,
update, delete     │   └── usePreferences.ts         # concerns, guardians, theme from DB     ├──
components/     │   ├── Nav.tsx                  # Main nav + skip-to-content link     │   ├──
ThemeToggle.tsx     │   ├── Toast.tsx     │   ├── SkeletonList.tsx          # Animated pulse,
respects prefers-reduced-motion     │   ├── AlertCard.tsx             # Title + all badges + Threat
DNA thumbnail     │   ├── AlertForm.tsx             # Shared: create (paste+manual) and edit     │
├── FilterBar.tsx             # All filters incl. affects-me + text search     │   ├──
SafetyPulse.tsx           # "You're caught up" / "N things need attention"     │   ├── TrendChart.tsx
# Recharts BarChart from DB data     │   ├── ThreatQuery.tsx           # AI Call 2 — natural language
query box     │   ├── DigestView.tsx            # Filtered list + "Directly affects you" section
│   ├── FocusMode.tsx             # One-at-a-time, Next/Done, progress indicator     │   ├──
CategoryChecklist.tsx     # 1-2-3 steps + local checkboxes     │   ├── ThreatDNA.tsx             #
SVG fingerprint — size prop (sm/lg)     │   ├── AffectsMe.tsx             # "This Affects Me" toggle
button     │   ├── SimilarAlerts.tsx         # 1-2 similar alert cards on create page     │   ├──
ReadAloud.tsx             # speechSynthesis wrapper     │   └── GuardianCircle.tsx        # Trusted
contacts CRUD + share actions     ├── app/     │   ├── layout.tsx                # Root layout,
ThemeProvider     │   ├── page.tsx                  # Redirect → /alerts     │   ├── alerts/     │
│   ├── page.tsx              # List page     │   │   └── [id]/     │   │       ├── page.tsx
# Detail page     │   │       └── edit/page.tsx     # Edit page     │   ├── digest/page.tsx     │
├── create/page.tsx     │   ├── settings/page.tsx     │   ├── shared/     │   │   ├──
digest/[token]/page.tsx     │   │   └── alert/[id]/page.tsx     │   └── api/     │       ├──
alerts/route.ts       # GET (list+filter) + POST (create)     │       ├── alerts/[id]/route.ts  # GET
+ PATCH + DELETE     │       ├── preferences/route.ts  # GET + PATCH     │       ├──
guardians/route.ts    # GET + POST     │       ├── guardians/[id]/route.ts # DELETE     │       ├──
categorize/route.ts   # AI Call 1 proxy     │       ├── query/route.ts        # AI Call 2 proxy     │
└── feeds/     │           ├── cisa/route.ts     # Fetch + normalise + upsert to DB     │
├── phishtank/route.ts     │           └── nvd/route.ts     └── tests/         ├──
  mocks  /prisma.ts       # mockDeep<PrismaClient>         ├── happyPath.test.ts         ├──
edgeCase.test.ts         ├── fallback.test.ts         └── emptyState.test.ts
```

§10 Execution Order (~13 Hours)

□ = Non-negotiable. Never cut. □ = Build after core. □ = Bonus if ahead.

	Phase	Time	Tasks
□	**Phase 0 Setup**	**30 min**	npx create-next-app (TypeScript + Tailwind + App Router). npm install zustand recharts uuid. npm install -D prisma @prisma/client vitest @testing-library/react jsdom jest-mock-extended. npx prisma init. Create .env + .env.example with both ANTHROPIC_API_KEY and DATABASE_URL. Create src/types/alert.ts.
□	**Phase 1 DB + Schema + Seed**	**75 min**	Write prisma/schema.prisma (Alert + UserPreference + Guardian models with all indexes). npx prisma db push. Create prisma/seed.ts — read data/alerts.json, prisma.alert.createMany({ skipDuplicates: true }). Create data/alerts.json (40-50 rows) + data/category_checklists.json. src/lib/db.ts — PrismaClient singleton. Build all 5 DB API routes: /api/alerts (GET+POST), /api/alerts/[id] (GET+PATCH+DELETE), /api/preferences (GET+PATCH), /api/guardians (GET+POST), /api/guardians/[id] (DELETE). Build src/store/useStore.ts — Zustand with optimistic update pattern. Build src/lib/fingerprint.ts — deterministic SVG generator.
□	**Phase 2 Real Data Layer**	**45 min**	Build /api/feeds/cisa/route.ts — fetch CISA KEV, normalise to Alert schema, upsert to DB via prisma.alert.upsert({ where: { id }, create, update }), cache 1hr. Build /api/feeds/phishtank/route.ts — same pattern. Build /api/feeds/nvd/route.ts — CVSS score → severity mapping. Build src/hooks/useAlerts.ts — fetch /api/alerts with query params, return typed Alert[].
□	**Phase 3 Core UI Shell**	**45 min**	Root layout.tsx with ThemeProvider (reads theme from /api/preferences on mount). Nav.tsx — Alerts | Digest | + Create | Settings links + skip-to-content. ThemeToggle.tsx. SkeletonList.tsx — 5 animated pulse cards, respect prefers-reduced-motion. Dark mode via Tailwind class strategy. All page routes stubbed.
□	**Phase 4 List + Filter + Safety Pulse**	**60 min**	AlertCard.tsx — title + category/severity/source badges + ThreatDNA fingerprint thumbnail (24x24) + date + resolved state + one CTA. ThreatDNA.tsx — renders deterministic SVG via generateFingerprint(). SafetyPulse.tsx — counts from DB: 0 active = green "caught up", N active = amber "N need attention", X affects_me = amber sub-line. TrendChart.tsx (Recharts) — query DB for last 14 days, stacked severity bars, collapsible. FilterBar.tsx — all filters including affects_me toggle + text search (debounced 300ms) + clear all. List page — mount → initStore() → fetch /api/feeds/* → upsert to DB → fetch /api/alerts → render. Empty state. "Last checked" timestamp.
□	**Phase 5 Create (Both Paths) + Similar Alerts**	**75 min**	AlertForm.tsx — shared component. Paste tab: textarea → "Categorize" button → loading → AI Call 1 (POST /api/categorize) → prefill all fields → confidence badge → "Verify" warning if low → SimilarAlerts.tsx (findSimilarAlerts() against DB alerts) → user edits → Save → POST /api/alerts → navigate to detail. Manual tab: all fields, inline validation on blur + submit validation. Validation: title (3-100), description (10-1000), category + severity (required), date (required, not future), location (optional, max 50). src/lib/similarity.ts — keyword overlap scoring against DB corpus.
□	**Phase 6 AI Call 1 + Fallback Engine**	**60 min**	/api/categorize/route.ts — POST handler, calls Anthropic API with system prompt, response_format json_object, returns parsed JSON, throws on failure. src/lib/ai.ts — getStructuredCategory(text) wrapper. src/lib/fallback.ts — fallbackCategorize(): keyword map per category, urgency regex, confidence always "low", reason: "Matched keywords: ...". "Using fallback" banner in UI when AI fails. Test fallback.ts in isolation as part of Phase 10.

	Phase	Time	Tasks
☐	**Phase 7 Detail Page + Update + This Affects Me**	**50 min**	Detail page /alerts/[id]: large ThreatDNA (64x64). Title + all badges. AI summary (calm style). "Why this category?" collapsible (reason field). Full description. CategoryChecklist — 1-2-3 from category_checklists.json, local checkboxes. AffectsMe.tsx — toggle button CISA/NVD only, PATCH /api/alerts/[id] with affects_me. Suggested action. ReadAloud.tsx — speechSynthesis. Action row: "Mark Resolved"/"Reopen" (PATCH DB) + "Edit" + "Share this alert". External link for CISA/NVD. Edit page — reuse AlertForm pre-populated from DB, Save → PATCH, Cancel → back.
☐	**Phase 8 Digest + My Concerns + Trend Chart**	**60 min**	Settings/onboarding — first-time OR from Settings: "What matters to you?" multi-select pills → PATCH /api/preferences. Digest page — "Directly affects you" section (affects_me=true from DB) pinned top. "For you" tab (filter by concerns from DB). "All active" tab. Focus Mode toggle → FocusMode.tsx (one card, Next/Mark Done/Back, "2 of 7"). "Share digest" → read-only link. "Read digest aloud" → speechSynthesis queue. TrendChart toggle: by severity | by category | by source (all from DB).
☐	**Phase 9 AI Call 2 + Location + A11y**	**45 min**	ThreatQuery.tsx — NL input, example prompts shown. /api/query/route.ts — build context from prisma.alert.findMany({ orderBy: created_at desc, take: 20 }), send to Anthropic (200 token max). Fallback: "Search unavailable. Browse alerts below." Location filter — prisma.alert.findMany({ distinct: [location] }) for dropdown options. Full A11y pass: skip-to-content, focus traps, aria-live, aria-labels, WCAG AA contrast, prefers-reduced-motion final check.
☐	**Phase 10 Guardian Circle + Share**	**45 min**	GuardianCircle.tsx — GET /api/guardians on mount, POST to add, DELETE to remove. Share flows: "Share digest" generates token → /shared/digest/[token], "Share alert" → /shared/alert/[id], copy to clipboard + "Copied!" toast. "Copy status message" → "I have N alerts (X directly affect me). View: [link]". Read-only shared pages — GET alert(s) from DB, no nav/edit, "view only" header.
☐	**Phase 11 Tests**	**45 min**	src/tests/__mocks__/prisma.ts — mockDeep<PrismaClient>() from jest-mock-extended. happyPath.test.ts — create alert (manual) → mock POST → appears in list → mock PATCH → mark resolved. edgeCase.test.ts — empty paste → validation error, no API call; title < 3 → inline error; future date → inline error. fallback.test.ts — mock /api/categorize to throw → fallbackCategorize() called → category = Phishing for phishing text → "Using fallback" banner → confidence = "low" → "Verify" warning. emptyState.test.ts — mock GET /api/alerts returns [] → friendly empty state → "Clear filters" button shown.
☐	**Phase 12 Deploy + Docs + Video**	**45 min**	neon.tech → New Project → copy DATABASE_URL. npx prisma db push (against Neon). npx prisma db seed. Vercel dashboard → add ANTHROPIC_API_KEY + DATABASE_URL env vars. package.json build: "prisma generate && next build". git push → Vercel autodeploy → copy live URL. Fill README. Write DESIGN.md. Record 5-7 min video.

Total: ~13 hours. If behind → cut Phase 10 (Guardian/Share) first, then trim Phase 9 to A11y only. NEVER cut Phases 0-7, 11, 12.

§11 Tests (4 Required)

Test File	What It Covers	Steps
happyPath.test.ts	Full create → list → update flow with mocked DB	1. Render CreatePage (manual tab) 2. Fill all fields with valid data 3. Submit → mock POST /api/alerts → success 4. Navigate to /alerts 5. Mock GET /api/alerts returns new alert 6. Assert alert in list with Threat DNA 7. Click → detail page 8. Click "Mark Resolved" → mock PATCH 9. Assert resolved state
edgeCase.test.ts	Invalid input → correct errors, no DB/API calls	1. Submit paste tab with empty textarea → assert validation error, no fetch 2. Submit title < 3 chars → assert inline error on title field 3. Submit future date → assert inline error on date field 4. Assert zero API calls in all three cases
fallback.test.ts	AI failure → fallback runs, UI reflects it	1. Mock /api/categorize to throw network error 2. Paste phishing text, click "Categorize" 3. Assert fallbackCategorize() called 4. Assert category = "Phishing" 5. Assert "Using fallback" banner visible 6. Assert confidence badge = "low" 7. Assert "Verify this classification" warning visible
emptyState.test.ts	No DB results → friendly empty state + recovery	1. Mock GET /api/alerts returns [] 2. Apply additional filter (category=Other) 3. Assert "No alerts match your filters" message 4. Click "Clear filters" 5. Mock returns alerts again 6. Assert list reappears

Prisma Mock Setup

```
// src/tests/__mocks__/prisma.ts import { PrismaClient } from '@prisma/client' import { mockDeep,
mockReset } from 'jest-mock-extended'  export const prismaMock = mockDeep<PrismaClient>()
beforeEach(() => { mockReset(prismaMock) })  // In API route tests, mock the db module: //
vi.mock('@/lib/db', () => ({ default: prismaMock }))
```

§12 Deliverables Checklist

	Deliverable	Details
☐	**Working prototype (deployed)**	Vercel URL in README. All flows functional. DB live on Neon.
☐	**data/alerts.json**	40-50 synthetic alerts, FTC-inspired, all Alert fields populated including affects_me=false
☐	**data/category_checklists.json**	1-2-3 steps for: Phishing, Scam, Imposter, Data breach, Local safety, CVE
☐	**prisma/schema.prisma**	Alert + UserPreference + Guardian models — committed to repo
☐	**prisma/seed.ts**	Seeds data/alerts.json to Neon DB — npx prisma db seed
☐	**DESIGN.md**	Problem, solution, architecture diagram, AI design decisions, DB schema rationale, future work
☐	**README.md (filled template)**	Candidate name, quick start, DB setup, live demo URL, AI disclosure, tradeoffs, limitations
☐	**.env.example**	ANTHROPIC_API_KEY + DATABASE_URL (placeholders only, committed)
☐	**5-7 min screen recording**	Problem → solution → real data + DB → AI + fallback → unique features demo → tests → learnings

README Template (Fill Before Submitting)

```
# Community Guardian  Candidate Name: [Your name] Scenario: Option 3 — Community Safety & Digital
Wellness Estimated Time: ~13 hours Live Demo: [Vercel URL]  ## Quick Start Prerequisites: Node 18+
1. Clone the repo 2. cp .env.example .env  →  fill in ANTHROPIC API KEY and DATABASE URL 3. npm
install 4. npx prisma db push 5. npx prisma db seed 6. npm run dev  Test: npm run test  ## Real Data
Sources CISA KEV:   https://www.cisa.gov/known-exploited-vulnerabilities-catalog PhishTank:
https://phishtank.org/developer info.php NVD:        https://nvd.nist.gov/developers/vulnerabilities
Note: Official public APIs — no live site scraping. Static snapshot in /data for offline use and
rubric compliance.  ## AI Disclosure Used AI assistant: Yes (Claude) Verified by: Testing all
suggestions, reading every line of generated code One rejected suggestion: [specific example here]
## Tradeoffs Went over 8-10hr timebox deliberately: added full DB (Neon+Prisma) for production-grade
persistence — noted as conscious architectural choice. Cut: Real encryption for Guardian Circle
(read-only links sufficient) Cut: User authentication (out of scope) Would build next: Real-time
WebSocket threat feed, PWA push notifications,   ML-based severity scoring trained on historical data
Known limitations: PhishTank rate limits on free tier;   NVD rate limits without API key (100
req/day)
```

Video Script (5-7 min)

Timestamp	Content
0:00 – 0:30	Problem: alert fatigue, scattered safety info, no actionable steps — information overload leads to paralysis
0:30 – 1:00	Solution overview: real threat intelligence (CISA, PhishTank, NVD) + AI filtering + calm actionable UX + full PostgreSQL persistence

Timestamp	Content
1:00 – 2:00	Demo: list view → Safety Pulse → real CISA/NVD data with source badges + Threat DNA fingerprints → Trend Chart → filters
2:00 – 3:00	Demo: create alert — paste phishing email → AI categorizes → confidence badge → Similar Alerts from DB → save → persisted
3:00 – 3:30	Demo: fallback — disconnect API → same paste → keyword fallback → "Using fallback" banner → "Verify" warning
3:30 – 4:15	Demo: detail page → Threat DNA hero → checklist → "This Affects Me" toggle → DB persisted → Safety Pulse updates
4:15 – 5:00	Demo: Natural language query — "What CVEs affect me this week?" → Claude reads DB context → calm answer
5:00 – 5:30	Demo: 4 tests running — all passing — brief explanation of each
5:30 – 6:00	Architecture: Neon + Prisma + optimistic Zustand cache — why full DB over localStorage
6:00 – 6:30	Tradeoffs + what's next + why this aligns with PAN's core mission

§13 Quick Reference

Phase Priority

	Phase	Time	Cut Priority
□	0. Setup	30m	NEVER CUT
□	1. DB + Schema + Seed	75m	NEVER CUT
□	2. Real Data Layer	45m	NEVER CUT
□	3. Core UI Shell	45m	NEVER CUT
□	4. List + Filter + Safety Pulse	60m	NEVER CUT
□	5. Create + Similar Alerts	75m	NEVER CUT
□	6. AI Call 1 + Fallback	60m	NEVER CUT
□	7. Detail + Update + This Affects Me	50m	NEVER CUT
□	8. Digest + My Concerns + Chart	60m	Trim: skip Focus Mode if tight
□	9. AI Call 2 + Location + A11y	45m	Trim: skip NL Query, keep A11y
□	10. Guardian Circle + Share	45m	CUT FIRST if behind
□	11. Tests	45m	NEVER CUT
□	12. Deploy + Docs + Video	45m	NEVER CUT

Total: ~13 hours. Timebox noted in README as deliberate architectural choice.

Install Commands

```
# Create project npx create-next-app@latest community-guardian \   --typescript --tailwind --app --
src-dir cd community-guardian  # Runtime npm install zustand recharts uuid @prisma/client  # Dev npm
install -D prisma vitest @testing-library/react   @testing-library/jest-dom jsdom jest-mock-extended
# Init Prisma npx prisma init  # After writing schema.prisma + setting DATABASE URL npx prisma db
push npx prisma db seed npx prisma generate  # Dev server npm run dev  # Tests npm run test  # Deploy
npx vercel --prod
```

package.json Additions

```
"scripts": {   "build": "prisma generate && next build",   "test": "vitest" }, "prisma": {   "seed":
"ts-node prisma/seed.ts" }
```